structures & *Linked List*

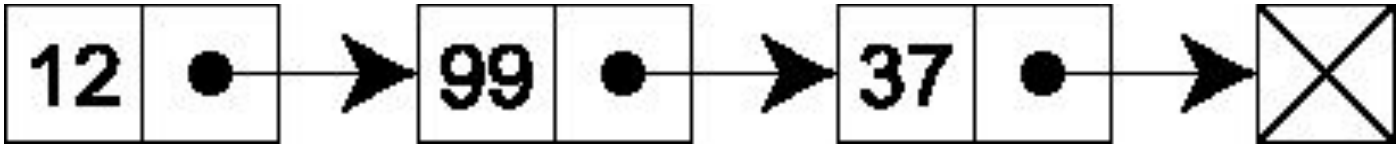
Introduction to Linked List

What is a linked list?

linked list is a data structure that can store a sequence (collection) of data records.

- *In each record there is a field that contains a reference (i.e., a link) to the next record in the sequence (self referential structures).*

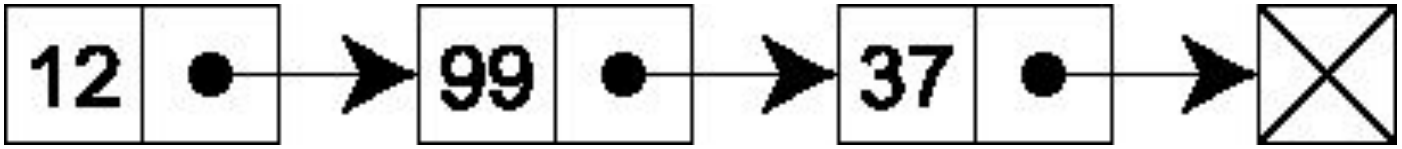
Linked List



A linked list whose nodes contain two fields:
-an integer value and
-a link to the next node

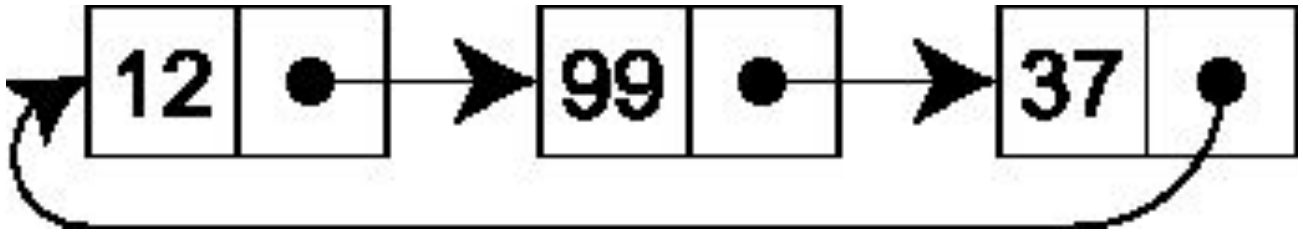
Singly-linked list (Linear or open Linked list)

- In the last node of a list, the link field often contains a **null** reference, a special value that is interpreted by programs as meaning "there is no such node".



Circular linked list

- make the last node's next pointer point to the first node of the list;
  - in that case the list is said to be **circular** or **circularly linked**;



Array Review

- It's the **most** common data structure used to store collections of elements.
- Most programming Languages provide the handy **[ ] subscript notation** to access any element by its index (offset) number.

Array Review

```
int scores[100];
scores[0] = 1;
scores[1] = 2;
scores[2] = 3;
```

- The entire array is allocated as one block of memory.
- Each element in the array gets its own space in the array.
- Any element can be accessed directly using the subscript notation []

Address	Content	Subscript not.
4000		a[0]
4004		a[1]
4008		a[2]
4012		a[3]
..		
..		
..		
..		
..		
..		
..		a[98]
4396	?	a[99]

Array Review

```
int scores[100];
scores[0] = 1;
scores[1] = 2;
scores[2] = 3;
```

- Once the array is set up, access to any element is easy & fast with the subscript [] notation.

Address	Content	Subscript not.
4000		a[0]
4004		a[1]
4008		a[2]
4012		a[3]
..		
..		
..		
..		
..		
..		
..		a[98]
4396	?	a[99]

Disadvantages of Arrays

```
int scores[100];
scores[0] = 1;
scores[1] = 2;
scores[2] = 3;
```

Address	Content	Subscript not.
4000		a[0]
4004		a[1]
4008		a[2]
4012		a[3]
..		
..		
..		
..		
..		
..		
..		a[98]
4396	?	a[99]

1. The size of the array is fixed. It can not resize as it is statically allocated.
2. Because of (1), the most convenient thing for programmers to do is to allocate arrays which seem “LARGE ENOUGH”! Although it is convenient, it has two more disadvantages-
    i. If most of the time there are only 20 to 30 elements in the array, and 70% of the space in the array is wasted.

Disadvantages of Arrays

```
int scores[100];
scores[0] = 1;
scores[1] = 2;
scores[2] = 3;
```

1. The size of the array is fixed. It can not resize as it is statically allocated.
2. Because of (1), the most convenient thing for programmers to do is to allocate arrays which seem "LARGE ENOUGH"! Although it is convenient, it has two more disadvantages-
    ii. If the program ever needs to process more than 100 scores, the code breaks!

Address	Content	Subscript not.
4000		a[0]
4004		a[1]
4008		a[2]
4012		a[3]
..		
..		
..		
..		
..		
..		
..		a[98]
4396	?	a[99]

Disadvantages of Arrays

```
int scores[100];
scores[0] = 1;
scores[1] = 2;
scores[2] = 3;
```

Address	Content	Subscript not.
4000		a[0]
4004		a[1]
4008		a[2]
4012		a[3]
..		
..		
..		
..		
..		

3. Inserting new elements at the front is potentially expensive because existing elements need to be shifted over to make room.

Linked lists have their own strengths & weaknesses, but they happen to be strong where arrays are weak!

Dynamic Memory Allocation

```
#include<stdio.h>

int main(){
    int a[100]; //static a

    return 0;
}
```

Address	Content	Access name
4000	?	a[0]
4004	?	a[1]
4008	?	a[2]
4012	?	a[3]
4016	?	a[4]
..		
4396	?	a[99]

Static allocation:

1. Allocate storage during the declaration
2. The size can not be modified during program execution
3. The size of the static array should be a constant.

Dynamic Memory Allocation

```
#include<stdio.h>
#include<stdlib.h>

int main(){
    int *a;
    a = malloc( 100 * sizeof(int) );

    return
}
```

Problem is:
malloc() function returns an address, but
You have to cast the type of the address!

Dynamic Memory Allocation

Address	Content	Access name
4000	?	a[0]
4004	?	a[1]
4008	?	a[2]
4012	?	a[3]
4016	?	a[4]
..		
4396	?	a[99]

```
#include<stdio.h>
#include<stdlib.h>

int main(){
    int *a;
    a = (int *)malloc( 100 * sizeof(int) );

    return 0;
}
```

Here, 100 can be variable, like

```
int x;
scanf("%d",&x);
a = (int *)malloc(x*sizeof(int));
```

Dynamic Memory Allocation

```
int main(){
    //EmployeeInfo p[50]; //Static allocation

   //Dynamic Allocation
    EmployeeInfo *p =
   (EmployeeInfo *)malloc(50 * sizeof(EmployeeInfo));

    return 0;
}
```

Self Referential structures

- It is sometimes desirable to include within a structure one member that is a pointer to the parent structure type.
- For example:

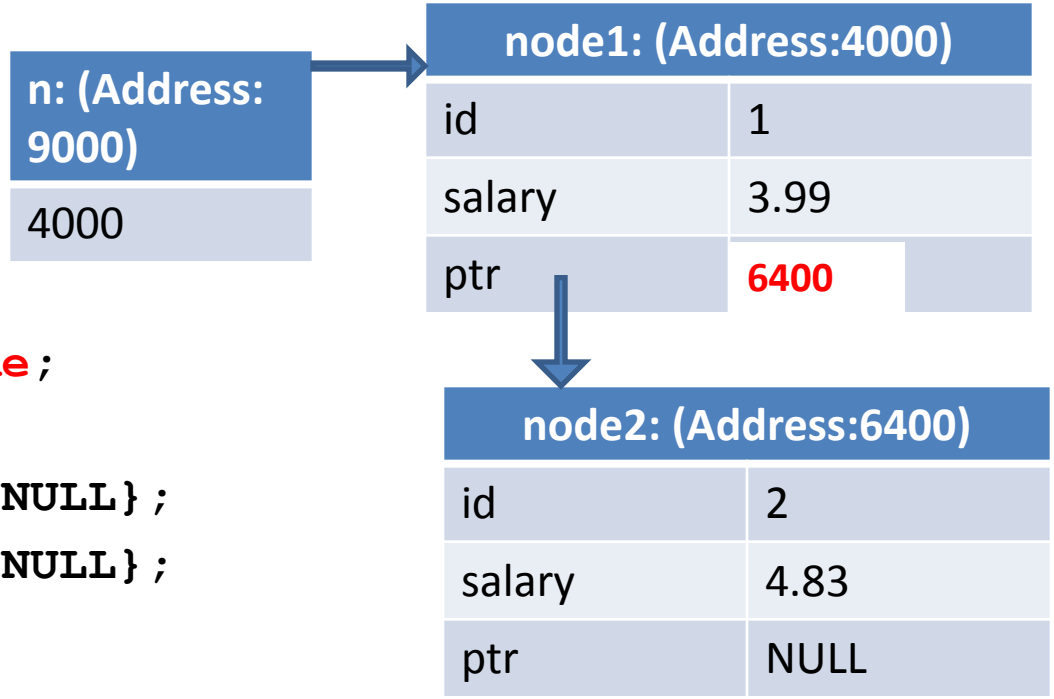
```
struct Node{
    int id;
    double salary;
    struct Node *ptr;
};
```

Self Referential structures

```
struct Node{
    int id;
    double salary;
    struct Node *ptr;
};
typedef struct Node Node;
int main(){
    Node node1={1,3.99,NULL};
    Node node2={2,4.83,NULL};
    Node *n = &node1;

    printf("id: %d\n",n->id);
    printf("salary: %lf\n",n->salary);

    node1.ptr = &node2;
    printf("id: %d\n",node1.ptr->id);
    printf("salary: %lf\n",node1.ptr->salary);
```




```
id: 1
salary: 3.990000
```

```
id: 2
salary: 4.830000
```

Basic components of Linked List

- Element/Node of a List
  - Each record of a linked list is often called an **element** or **node**.
- Fields in a Node: (i) data (ii) reference/pointer

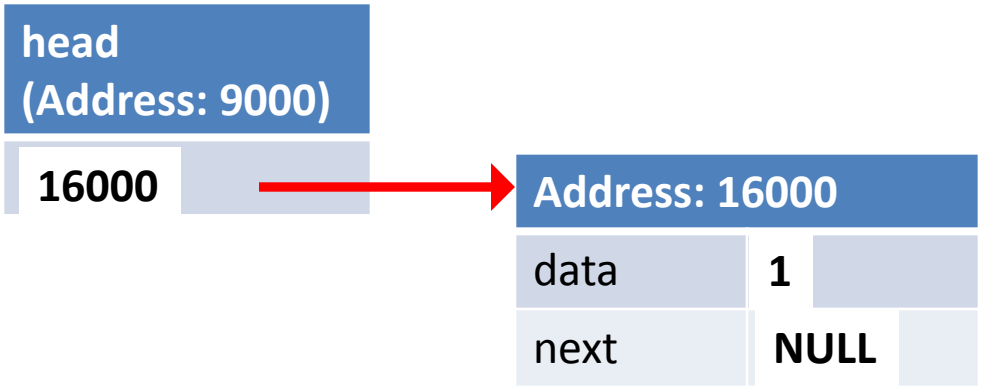
```
struct Node{
    int data;
    struct Node *next;
};
typedef struct Node Node;
```

Creating a linked list

```
//initially we have an empty list
Node *head = NULL;
```

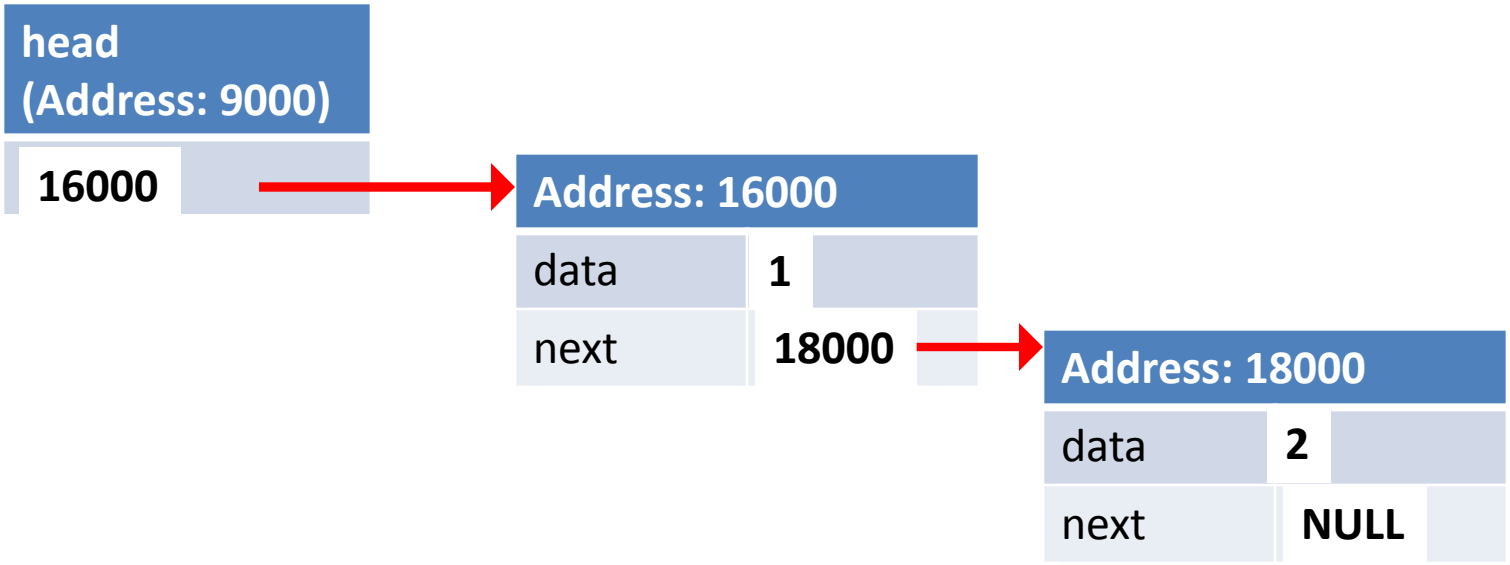
Creating a linked list

```
//Create a new node
head = (Node *)malloc(sizeof(Node));
head->data = 1;
head->next = NULL;
```



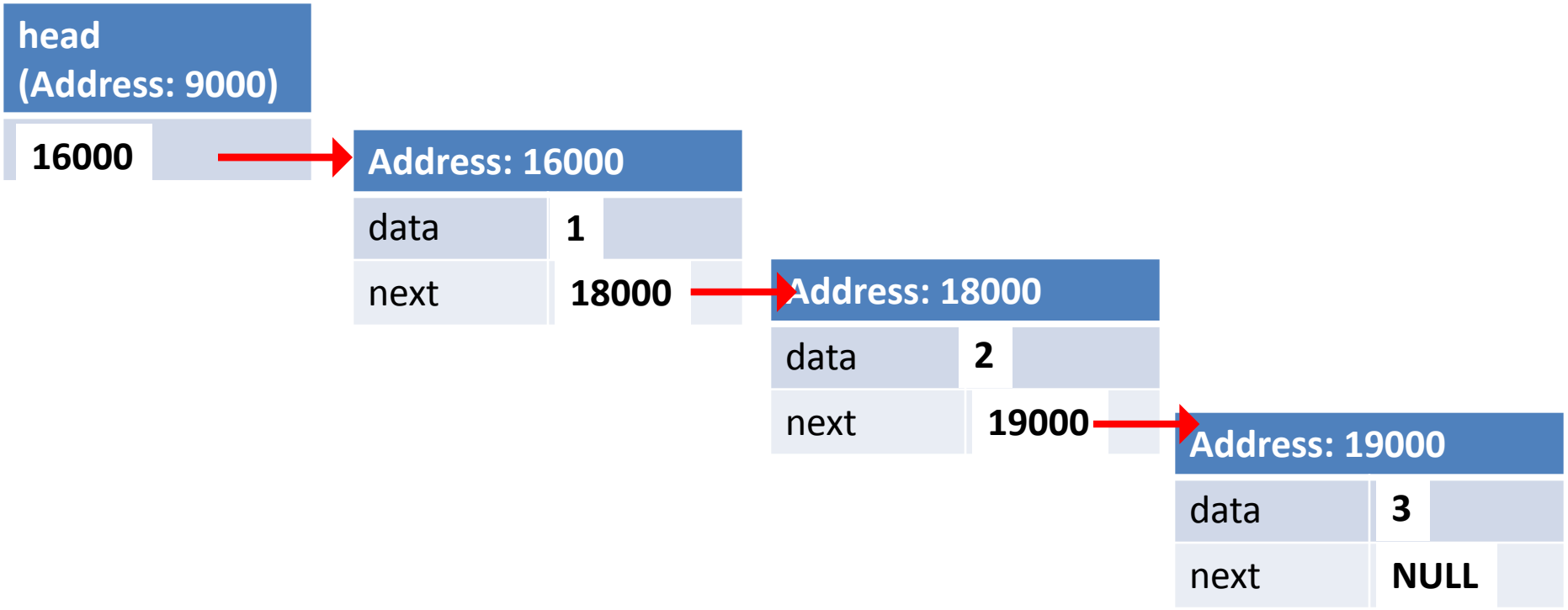
Creating a linked list

```
//Another node
head->next = (Node *)malloc(sizeof(Node));
head->next->data = 2;
head->next->next = NULL;
```



Creating a linked list

```
//Another node
head->next->next = (Node *)malloc(sizeof(Node));
head->next->next->data = 3;
head->next->next->next = NULL;
```



Let us create a structure for Node

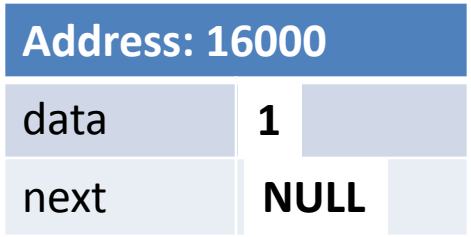
- Basic node style

```
// A Linked List Node
struct Node
{
   int data;            // integer data
   struct Node* next;   // pointer to the next node
};
```


```
struct Node  *head = NULL;
```

Creating a new node

```
// allocate a new node in a heap and set its data
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    scanf("%d",&item);
    node->data=item;
    node->next = NULL;
```

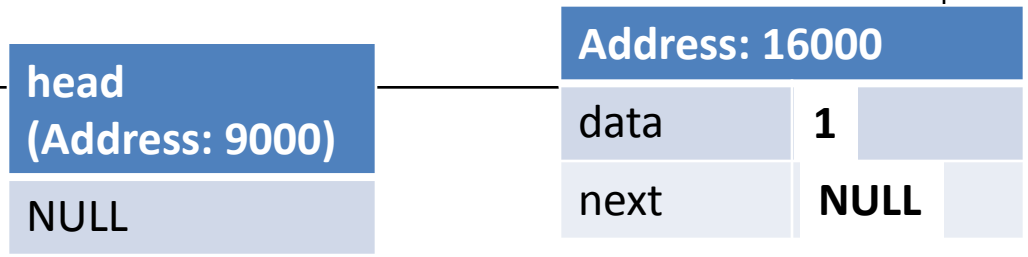


All together

```
// A Linked List Node
struct Node
{
   int data;            // integer data
   struct Node* next;   // pointer to the next node
};
```

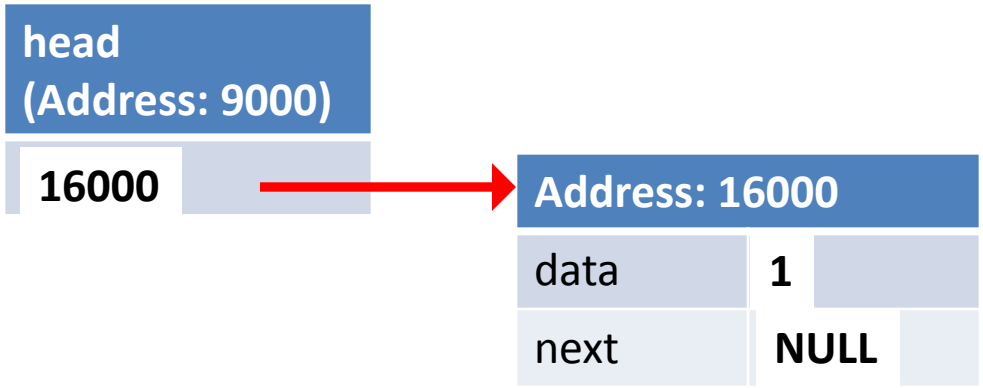
```
struct Node *head = NULL;
```

```
// allocate a new node in a heap and set its data
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    scanf("%d",&item);
    node->data=item;
    node->next = NULL;
```



Create the list

```
if(head == NULL)
  {
   head = node;
  }
```

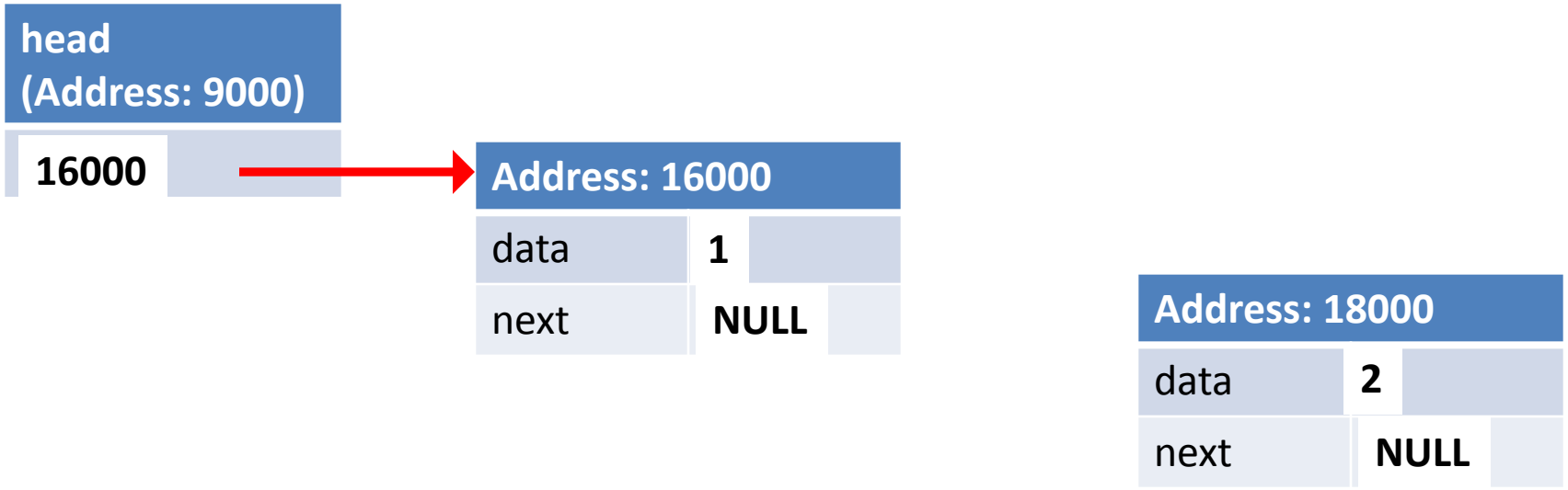


Summary

- Step 1 - Create a **new node** with given value.
- Step 2 - Check whether list is **Empty** (**head** == **NULL**)
- Step 3 - If it is **Empty** then, set **head** = **node**.

Creating another new node

```
// allocate a new node in a heap and set its data
   struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    scanf("%d",&item);
    node->data=item;
    node->next = NULL;
```

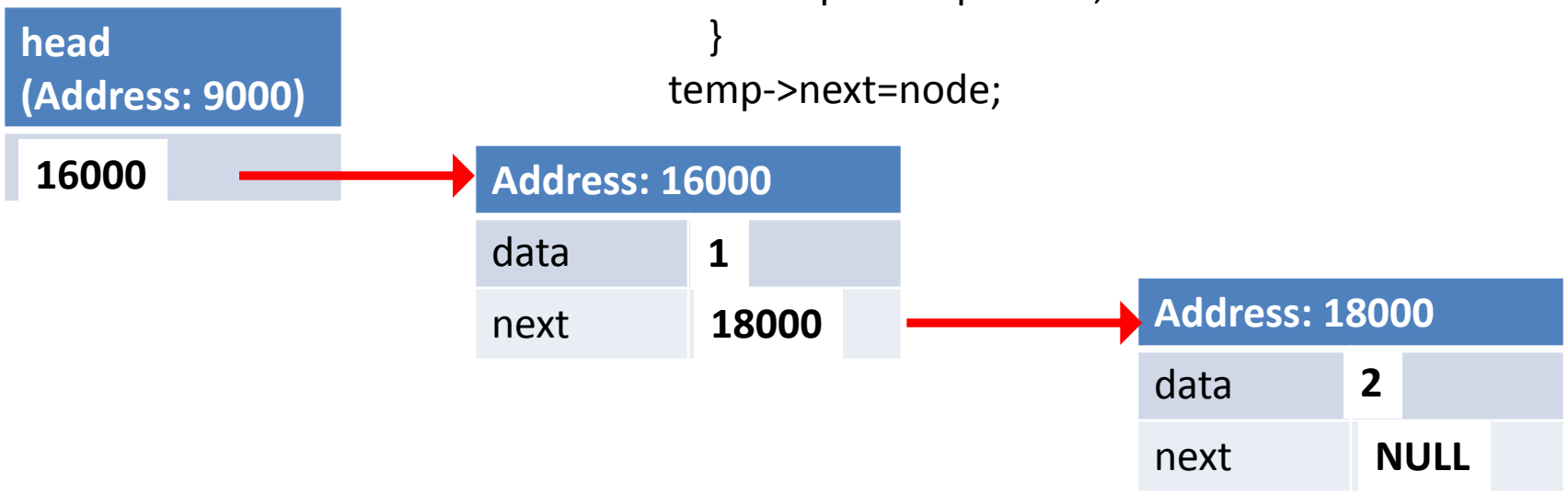


Creating another new node

Inserting At End of the list

- Step 1 - If it is **Not Empty** then, define a node pointer **temp** and initialize with **head**.
- Step 1 - Keep moving the **temp** to its next node until it reaches to the last node in the list (until **temp** → **next** is equal to **NULL**).
- Step 6 - Set **temp** → **next** = **node**.

```
struct Node* temp = head;
   while (temp->next!=NULL)
   {
      temp = temp->next;
   }
temp->next=node;
```



Print

- Step 1 - Check whether list is **Empty** (**head** == **NULL**)
- Step 2 - If it is **Empty** then, display **'List is Empty!!!'** and terminate the function.
- Step 3 - If it is **Not Empty** then, define a Node pointer **'temp'** and initialize with **head**.
- Step 4 - Keep displaying **temp** → **data** with an arrow (**--->**) until **temp!=NULL**

```
if(head == NULL)
  {
    printf("\nList is Empty\n");
  }
  else
  {
    struct Node *temp = head;
    printf("\n\nList elements are - \n");
    while(temp != NULL)
    {
        printf("%d --->",temp->data);
        temp = temp->next;
    }
}
```